



School of Science

Master's Degree in Data Science

Course: Optimization for Data Science

Optimization methods for clustering

Professors

Gennaro Auricchio
Francesco Rinaldi

Students (Group 11)

Miguel Cruz Real (2183365)
Reza Mahin Mohammadalizadeh (2181617)

Academic Year 2025–26

Contents

1	Introduction	1
2	Methods	1
2.1	Problem setup	1
2.2	The classical Frank–Wolfe method . .	2
2.3	The Frank–Wolfe gap	2
2.4	Away-Step Frank–Wolfe (AFW)	2
2.5	Pairwise Frank–Wolfe (PFW)	3
2.6	Application to the maximum-clique problem	3
2.7	Convergence guarantees	4
3	Datasets	4
3.1	Graph representation	4
3.2	Benchmark instances	5
3.3	Preprocessing	5
4	Practical results	5
4.1	Experimental setting	5
4.2	Performance metrics	5
4.3	Clique quality	5
4.4	Stability across starts	6
4.5	Computational time	6
4.6	Discussion	7
5	Conclusions	7

1 Introduction

Given a simple undirected graph, a *clique* is a subset of vertices that are pairwise adjacent, and the *maximum clique problem* (MCP) asks for a clique of the largest possible size. Despite its simple statement, the MCP is a fundamental combinatorial problem with applications in social network analysis, bioinformatics, telecommunication networks and scheduling, among others.

A classical line of work reshape the MCP as a continuous quadratic program over the probability simplex, following the Motzkin–Straus formulation. This formulation, however, admits spurious solutions from which no clique can be recovered; adding a suitable regularization term removes them and restores a one-to-one correspondence between the optimizers of the continuous problem and the (maximal and maximum) cliques of the graph. The resulting *2-regularized* formulation is the one we will be adopting in this work.

Because the feasible set is the simplex, the problem is naturally suited to the Frank–Wolfe family of *projection-free* first-order methods, whose iterates are sparse combinations of vertices and map directly onto candidate cliques. In this project we implement and compared the classical Frank–Wolfe method (FW) together with two variants, Away-Step Frank–Wolfe (AFW) and Pairwise Frank–Wolfe (PFW). Since the regularized objective is non-convex, each method is embedded in a multistart framework, and the three

are evaluated on three instances of the DIMACS maximum-clique benchmark, spanning different densities and clique sizes. We compare them in terms of the quality of the recovered cliques and of their computational cost.

The report is organized as follows: in the section 2 we present the regularized formulation and the three Frank–Wolfe variants, together with their application to the MCP and their convergence guarantees. In the section 3 we describe the benchmark instances and preprocessing. For the section 4 we report and discuss the numerical results, and we talk about our conclusions on section 5.

2 Methods

2.1 Problem setup

Let $G = (\mathcal{V}, \mathcal{E})$ be a simple undirected graph on $\mathcal{V} = \{1, \dots, n\}$ with adjacency matrix $A = (a_{ij})$, and let $\omega(G)$ denote the size of a maximum clique. The *maximum-clique problem* (MCP) asks for a clique of maximum cardinality and NP-hard. A classical continuous reformulation showed that:

$$\max_{x \in \Delta} x^\top A x = 1 - \frac{1}{\omega(G)}, \quad \Delta := \{x \in \mathbb{R}_{\geq 0}^n : \mathbf{1}^\top x = 1\}, \quad (1)$$

This quadratic problem, however, admits spurious solutions on the simplex from which no clique can be recovered. To remove them, Bomze [1] introduced a regularization term with this choice of parameters:

$$\Phi(x) := \frac{1}{2} \|x\|_2^2, \quad (2)$$

which yields the 2-norm regularization problem

$$\begin{aligned} \max_{x \in \Delta} F(x) &:= x^\top A x + \frac{1}{2} \|x\|_2^2 \\ &= \max_{x \in \Delta} x^\top M x, \\ M &:= A + \frac{1}{2} I. \end{aligned} \quad (3)$$

where A is the adjacency matrix of G . Thanks to the form of this problem, Frank–Wolfe methods can be fruitfully used to find maximal cliques. [3] For this choice, the local maximizers of (3) are in one-to-one correspondence with the maximal cliques of G , and the global maximizers with the maximum cliques. Given a maximizer x^* , the associated clique is recovered as its support, $C^* = \text{supp}(x^*)$. To align with the standard minimization form used by the conditional-gradient family, we equivalently minimize the negated objective over the same domain.

$$\min_{x \in \mathcal{D}} f(x), \quad f(x) := -x^\top M x, \quad \mathcal{D} := \Delta. \quad (4)$$

Its gradient and Hessian are

$$\nabla f(x) = -2Mx = -(2A + I)x, \quad \nabla^2 f(x) = -2M = -(2A + I). \quad (5)$$

Two features of (6) drive every design choice below. (i) *Non-convexity*. Since A is indefinite, so is $\nabla^2 f$;

thus f is neither convex nor concave, and first-order methods can only be guaranteed to reach a stationary (KKT) point of (6), i.e. a maximal clique. Therefore, we pursue global optimality through a multistart (Section 2.6). (ii) *Simplicial geometry*. The feasible set is the unit simplex, whose vertices are the canonical basis vectors $\{e_1, \dots, e_n\}$, each uniquely representing a distinct node in the graph. This makes the linear subproblems below essentially free and establish a direct mapping between the algorithmic iterates and subsets of the vertex set $\mathcal{V}$.

We solve (6) with the Frank–Wolfe method and its away-step and pairwise variants, which are *projection-free*: every iteration solves a linear minimization over Δ , to find the best vertex to move, instead of a Euclidean projection.

2.2 The classical Frank–Wolfe method

The original Frank–Wolfe method, is an iterative first-order optimization algorithm to solve quadratic programming problems with linear constraints. FW relies on a simpler linear optimization subproblem at each iteration. We consider a general constrained convex optimization problem with the form:

$$\min_{x \in \mathcal{D}} f(x) \quad (6)$$

- $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is a convex function with Lipschitz continuous gradient having constant $L > 0$.
- $D \in \mathbb{R}^n$ is a convex compact set.

First of all, it selects an initial feasible point $x^{(0)} \in D$. Then we proceed for a fixed number of iterations k . At each iteration, FW linearizes f at the current iterate $x^{(k)}$ and moves toward a minimizer of this linear model over D , returned by the *Linear Minimization Oracle* (LMO),

$$s^{(k)} \in \arg \min_{s \in D} \langle s, \nabla f(x^{(k)}) \rangle. \quad (7)$$

The solution $s^{(k)}$ identifies an extreme point of the feasible region D that is most aligned with the direction of steepest descent in the linearized objective. The LMO simply returns a vertex of D , that becomes the target for the next convex update. The new iterate is obtained by a line search along the FW direction $d_k^{FW} := s^{(k)} - x^{(k)}$:

$$\begin{aligned} x^{(k+1)} &= x^{(k)} + \gamma_k d_k^{FW} \\ \gamma_k &\in \arg \min_{\gamma \in [0,1]} f(x^{(k)} + \gamma d_k^{FW}). \end{aligned} \quad (8)$$

As f is quadratic, the line search is exact and closed form. Writing $\phi(\gamma) = f(x^{(k)} + \gamma d)$ for a feasible direction d (with $\mathbf{1}^\top d = 0$), one has $\phi(\gamma) = \phi(0) + \gamma \nabla f(x^{(k)})^\top d - \frac{\gamma^2}{2} c_d$ with curvature $c_d := d^\top (2A + I) d$. The minimizer over $[0, \gamma_{\max}]$ is

$$\gamma_k = \begin{cases} \text{clip}_{[0, \gamma_{\max}]} \left(\frac{\nabla f(x^{(k)})^\top d}{c_d} \right), & c_d < 0, \\ \gamma_{\max}, & c_d \geq 0, \end{cases} \quad (9)$$

with $\gamma_{\max} = 1$ for a FW step. The branch $c_d \geq 0$ is a direct fingerprint of non-convexity: along such directions ϕ is concave and the optimum is attained at the boundary of the step interval. A classical FW step rescales all active weights, $\alpha_s^{(k+1)} = (1 - \gamma_k) \alpha_s^{(k)} + \gamma_k$ for $s = s^{(k)}$ and $\alpha_v^{(k+1)} = (1 - \gamma_k) \alpha_v^{(k)}$ otherwise.

Algorithm 1: Classical Frank–Wolfe Method (Exact Line Search)

- 1: Let $x^{(0)} \in \mathcal{D}$ be an initial feasible point
 - 2: **for** $k = 0, 1, 2, \dots, K$ **do**
 - 3: Compute gradient $\nabla f(x^{(k)})$
 - 4: $s^{(k)} \leftarrow \arg \min_{s \in \mathcal{D}} \langle s, \nabla f(x^{(k)}) \rangle$ ▷ Linear Minimization Oracle
 - 5: $d_k^{FW} \leftarrow s^{(k)} - x^{(k)}$ ▷ FW direction
 - 6: Calculate $\gamma_k \in [0, 1]$ using Eq. 9 ▷ Exact line search
 - 7: $x^{(k+1)} \leftarrow x^{(k)} + \gamma_k d_k^{FW}$ ▷ Update step
 - 8: **end for**
-

2.3 The Frank–Wolfe gap

The *Frank–Wolfe gap* measures how much the objective is predicted to decrease, according to the linear model of f at $x^{(k)}$, when moving from the current iterate toward the vertex $s^{(k)}$ returned by the LMO, and is computed as:

$$\begin{aligned} g_k &:= \langle -\nabla f(x^{(k)}), s^{(k)} - x^{(k)} \rangle \\ &= \max_{s \in D} \langle -\nabla f(x^{(k)}), s - x^{(k)} \rangle \geq 0. \end{aligned} \quad (10)$$

A point with $g_k = 0$ satisfies the first-order optimality conditions of (6): we stop when $g_k \leq \varepsilon$. In the convex case g_k also upper-bounds the primal suboptimality $f(x^{(k)}) - f^*$ and serves as a duality gap [4]; in our non-convex case it loses the interpretation we have talked about right before and must be read as a *measure of stationarity* rather than of suboptimality.

2.4 Away-Step Frank–Wolfe (AFW)

Classical FW can only move mass toward a vertex; it can never remove mass from a vertex already in use. This is what causes its slow zig-zag when the solution lies on a face of Δ . AFW fixes this issue by adding an *away direction*, which pulls mass out of a badly chosen vertex. The idea is, not only move towards a new "good" vertex, as in classical FW, but also to move away from an "old" vertex that is currently "bad" with respect to the objective function. To do so, AFW keeps track of the vertex the current iterate is built from: it maintains an *active set* $S^{(k)}$ of vertices with positive weights $\{\alpha_v^{(k)}\}$ such that $x^{(k)} = \sum_{v \in S^{(k)}} \alpha_v^{(k)} v$. At each iteration it computes both the usual FW vertex $s^{(k)}$ from (7) and the *away vertex*, that is the active vertex that most increases f (the one we want to drop):

$$v^{(k)} \in \arg \max_{v \in S^{(k)}} \langle \nabla f(x^{(k)}), v \rangle, \quad d_k^A := x^{(k)} - v^{(k)}. \quad (11)$$

We will choose now, the direction (between the FW and the away) achieving the largest alignment with $-\nabla f(x^{(k)})$. If we choose the one of FW, the step uses $\gamma_{\max} = 1$; the away step uses $\gamma_{\max} = \alpha_{v^{(k)}}^{(k)} / (1 - \alpha_{v^{(k)}}^{(k)})$, the largest factor that keeps all weights nonnegative, so feasibility is preserved without any stronger oracle. If we choose the away step with $\gamma_k = \gamma_{\max}$, the weight of $v^{(k)}$ drops to zero, so it leaves the active set, such a drop step removes a vertex from the support. The complete procedure is summarized in Algorithm 2.

Algorithm 2: Away-Step Frank–Wolfe (AFW)

```

1: Let  $x^{(0)} \in \Delta$ , and  $S^{(0)} = \{x^{(0)}\}$ 
2: for  $k = 0$  to  $K$  do
3:   Let  $s^{(k)} = \text{LMO}(\nabla f(x^{(k)}))$  and  $d_k^{FW} = s^{(k)} - x^{(k)}$ 
4:   Let  $v^{(k)} \in \arg \max_{v \in S^{(k)}} \langle \nabla f(x^{(k)}), v \rangle$  and  $d_k^A = x^{(k)} - v^{(k)}$ 
5:   Let  $g_k := \langle -\nabla f(x^{(k)}), d_k^{FW} \rangle$ 
6:   if  $g_k \leq \varepsilon$  then
7:     return  $x^{(k)}$ 
8:   end if
9:   if  $\langle -\nabla f(x^{(k)}), d_k^{FW} \rangle \geq \langle -\nabla f(x^{(k)}), d_k^A \rangle$  then
10:     $d_k = d_k^{FW}$ ,  $\gamma_{\max} = 1$ 
11:  else
12:     $d_k = d_k^A$ ,  $\gamma_{\max} := \frac{\alpha_{v^{(k)}}^{(k)}}{1 - \alpha_{v^{(k)}}^{(k)}}$ 
13:  end if
14:  Line search:  $\gamma_k \in \arg \min_{\gamma \in [0, \gamma_{\max}]} f(x^{(k)} + \gamma d_k)$ 
15:  Update  $x^{(k+1)} = x^{(k)} + \gamma_k d_k$ 
16:  Update  $S^{(k+1)} = \{v \in \Delta : \alpha_v^{(k+1)} > 0\}$ 
17: end for

```

2.5 Pairwise Frank–Wolfe (PFW)

The pairwise variant, inspired by the MDM algorithm of Mitchell et al. [8], takes the idea of the away step one stage further: instead of choosing between moving toward the FW vertex or away from the bad one, it does both at once, transferring mass directly from the away vertex $v^{(k)}$ onto the FW vertex $s^{(k)}$. The pairwise direction is therefore

$$d_k^{PFW} = s^{(k)} - v^{(k)}, \quad \gamma_{\max} = \alpha_{v^{(k)}}^{(k)}, \quad (12)$$

and a step along it leaves every other weight untouched, simply moving an amount γ_k from one vertex to the other:

$$\alpha_{v^{(k)}}^{(k+1)} = \alpha_{v^{(k)}}^{(k)} - \gamma_k, \quad \alpha_{s^{(k)}}^{(k+1)} = \alpha_{s^{(k)}}^{(k)} + \gamma_k. \quad (13)$$

The step size is capped at $\gamma_{\max} = \alpha_{v^{(k)}}^{(k)}$, the most that can be removed from $v^{(k)}$ before its weight would turn negative. The method coincides with Algorithm 2 once lines 9–13 are replaced. This update completes a natural progression across the three methods: a

classical FW step rescales all active weights at once, an away step redistributes the removed mass over the whole active set, and the pairwise step is fully localized, transferring weight from a single "bad" vertex onto a single "good" one. Therefore, PFW is highly effective at pruning irrelevant coordinates from the active set, whereas the away step spreads the freed mass over all active atoms and may need corrective steps later. The price guarantees are that the convergence guarantees of PFW are generally weaker than those of AFW, since the number of swap steps is hard to bound; in practice, however, PFW often matches or outperforms AFW, particularly when the solution is sparse. This makes it well suited to our setting, where the optimal support is a clique of size $\omega(G) \ll n$ and the iterates must collapse onto a low-dimensional face of Δ . The complete procedure is summarized in algorithm 3.

Algorithm 3: Pairwise Frank–Wolfe (PFW)

```

1: Let  $x^{(0)} \in \Delta$ , and  $S^{(0)} = \{x^{(0)}\}$ 
2: for  $k = 0$  to  $K$  do
3:   Let  $s^{(k)} = \text{LMO}(\nabla f(x^{(k)}))$  and  $d_k^{FW} = s^{(k)} - x^{(k)}$ 
4:   Let  $v^{(k)} \in \arg \max_{v \in S^{(k)}} \langle \nabla f(x^{(k)}), v \rangle$  and  $d_k^A = x^{(k)} - v^{(k)}$ 
5:   Let  $g_k := \langle -\nabla f(x^{(k)}), d_k^{FW} \rangle$ 
6:   if  $g_k \leq \varepsilon$  then
7:     return  $x^{(k)}$ 
8:   end if
9:    $d_k = d_k^{PFW} = s^{(k)} - v^{(k)}$ ,  $\gamma_{\max} = \alpha_{v^{(k)}}^{(k)}$ 
10:  Line search:  $\gamma_k \in \arg \min_{\gamma \in [0, \gamma_{\max}]} f(x^{(k)} + \gamma d_k)$ 
11:  Update  $x^{(k+1)} = x^{(k)} + \gamma_k d_k$ 
12:  Update  $S^{(k+1)} = \{v \in \Delta : \alpha_v^{(k+1)} > 0\}$ 
13: end for

```

2.6 Application to the maximum-clique problem

Specializing the three algorithms to (6) on the simplex makes their per-iteration cost essentially linear in the number of active vertices. The LMO is the coordinate scan; the away atom (11) is selected only within the (small) active set $S^{(k)}$, which is far cheaper than the global oracle; and the exact line search (9) requires a single quadratic curvature $c_d = d^\top (2A + I)d$. Throughout the run, $S^{(k)} = \text{supp}(x^{(k)})$ is a subset of $\mathcal{V}$, so the active set is literally a candidate vertex set and, at termination, the recovered clique is

$$C^* := \text{supp}(x^*). \quad (14)$$

Drop steps in AFW and the pruning steps of PFW thus translate directly into the removal of vertices from the current candidate.

Because (6) is non-convex, a single run converges to a maximal, but is not necessarily maximum, clique. To approximate $\omega(G)$ (size of the maximum clique) we embed each algorithm in a *multistart* framework.

Multistart framework: For every instance we perform $R = 100$ independent runs. The first run starts from the barycenter of the simplex, $x^{(0)} = \frac{1}{n}\mathbf{1}$; the remaining $R - 1$ runs start from points drawn uniformly at random from the simplex Δ . Each run applies the corresponding Frank–Wolfe variant (FW, AFW or PFW) with exact line search, iterating until the Frank–Wolfe gap satisfies $g_k \leq \varepsilon$ (with $\varepsilon = 10^{-6}$) or a maximum of 1000 iterations is reached. After optimization, the support of the final iterate is thresholded at $\frac{1}{2n}$ and a valid clique is extracted greedily (see Section 4 for details). We keep the largest clique found across the R runs, breaking ties by the value of the regularized objective. The same random seed, and therefore the same set of R starting points is used for FW, AFW and PFW, so the three methods are compared on an equal way. For the evaluation of the methods, we have used the DIMACS maximum-clique benchmark [5]: C125-9, brock200-2 and keller4, which span different densities and clique sizes; the corresponding adjacency matrices are stored in sparse format to exploit the sparsity of A in every gradient evaluation.

2.7 Convergence guarantees

We summarize the theory in the form relevant to a non-convex objective over a polytope, distinguishing what is *guaranteed* for (6) from the convex results that *motivate* the away-step and pairwise variants.

The starting point, common to both regimes, is the per-iteration progress of a conditional-gradient step. For an update $x^{(k+1)} = x^{(k)} + \gamma d_k$ with $\gamma \in [0, \gamma_{\max}]$, a second-order expansion of f gives

$$f(x^{(k+1)}) \leq f(x^{(k)}) - \gamma g_k + \frac{\gamma^2}{2} C_f, \quad (15)$$

where g_k is the Frank–Wolfe gap (10) and C_f is the *curvature constant* of f , which measures how much f departs from its linear model along feasible directions [4]. In our quadratic setting this curvature is exactly the quantity $c_d = d^\top (2A + I) d$ used in the exact line search (9). Inequality (15) holds without any convexity assumption and is the cornerstone of every convergence result below.

Non-convex guarantee (our setting). For a differentiable f with L -Lipschitz gradient over a compact convex set, the classical FW method drives the Frank–Wolfe gap to zero at the rate

$$\min_{0 \leq k \leq K} g_k \leq \frac{\max\{2h_0, LD^2\}}{\sqrt{K} + 1} = O\left(\frac{1}{\sqrt{K}}\right), \quad (16)$$

where D is the diameter of $\mathcal{D}$ and $h_0 = f(x^{(0)}) - f^*$ [6]. Since the gap measures stationarity, this certifies convergence to a first-order (KKT) point of (6), i.e. to a maximal clique, but *not* to a global optimum, and that is exactly why we resort to the multistart strategy of Section 2.6. An analogous stationarity rate holds for the away-step variant over the simplex [2]. These are the only guarantees that apply to our problem.

Convex theory. The away-step and pairwise variants were not designed for this rate but for a stronger one available in the convex case, which explains why we adopt them. When f is convex with L -Lipschitz gradient, classical FW satisfies the well-known sublinear bound

$$f(x^{(k)}) - f^* \leq \frac{2LD^2}{k+2} = O\left(\frac{1}{k}\right), \quad (17)$$

which is worst-case optimal for methods that add a single atom per iteration [4]. The bound is tight because FW cannot converge linearly in general: its iterates stall in a zig-zag when the solution x^* lies on a face of Δ . AFW and PFW were introduced precisely to remove this bottleneck. For objectives that are additionally strongly convex, both variants achieve *global linear convergence*,

$$f(x^{(k)}) - f^* \leq (1-\rho)^k (f(x^{(0)}) - f^*), \quad \rho \in (0, 1), \quad (18)$$

with ρ depending on the conditioning of f and on the geometry of the polytope [2, 7]. While these rates do not carry over verbatim to the non-convex problem (6), the geometric mechanism behind them (removing mass from suboptimal vertices to escape the zig-zag and to identify the optimal face quickly) is exactly what lets AFW and PFW recover the clique support C^* more efficiently than classical FW in our experiments (Section 4).

3 Datasets

The computational experiments were performed on three benchmark instances from the DIMACS collection for the maximum clique problem. These instances are standard test cases for clique algorithms and provide reference clique sizes, making them suitable for evaluating the quality of the solutions returned by the implemented Frank–Wolfe variants.

3.1 Graph representation

Each dataset is an undirected graph

$$G = (V, E),$$

where $V = \{1, \dots, n\}$ is the set of vertices and E is the set of edges. A clique is a subset $C \subseteq V$ such that every pair of distinct vertices in C is connected by an edge. The maximum clique problem consists of finding a clique with maximum cardinality.

In the continuous formulation used in this project, each graph is represented by its adjacency matrix

$$A \in \mathbb{R}^{n \times n},$$

where $A_{ij} = 1$ if vertices i and j are adjacent, and $A_{ij} = 0$ otherwise. This matrix is the input used by the objective function optimized by the Frank–Wolfe methods.

3.2 Benchmark instances

The selected benchmark instances belong to the DIMACS collection for the maximum clique problem [5] were downloaded from the Network Data Repository [9]. They are summarized in Table 1. In addition to the number of vertices, we report the number of edges and the graph density

$$\rho = \frac{2|E|}{n(n-1)}.$$

The column “best known” gives the reference clique size used to compare the quality of the recovered cliques.

Table 1: DIMACS graph instances used in the experiments.

Dataset	Vertices	Edges	Density	Best known
C125-9	125	6963	0.8985	34
brock200-2	200	9876	0.4963	12
keller4	171	9435	0.6491	11

The instance **C125-9** belongs to the random graph family of the DIMACS collection. It is a dense graph, with density close to 0.9, and has the largest reference clique size among the selected datasets. This makes it useful for evaluating whether the implemented methods can recover large clique candidates from the continuous solution.

The instance **brock200-2** belongs to the Brock family of benchmark graphs. These graphs were designed to be challenging for maximum clique algorithms, with hidden clique structures that are difficult to identify. In our experiments, this was the hardest instance: none of the implemented methods reached the best known clique size.

The instance **keller4** belongs to the Keller graph family. Keller instances are also known benchmark graphs for clique algorithms and are related to Keller’s conjecture. In our experiments, all implemented methods recovered a clique matching the reference value on this instance.

3.3 Preprocessing

The graph files were loaded from Matrix Market format and converted into binary adjacency matrices. During preprocessing, self-loops were removed by setting the diagonal entries to zero, all nonzero entries were binarized, and the matrices were symmetrized. This ensures that every instance is treated as a simple undirected graph, consistently with the maximum clique formulation used in the project.

4 Practical results

In this section, we present the computational results obtained by applying the three implemented Frank–Wolfe variants to the graph instances described in

Section 3. The methods compared are the classical Frank–Wolfe method (FW), the Away-Step Frank–Wolfe method (AFW), and the Pairwise Frank–Wolfe method (PFW).

4.1 Experimental setting

All methods were evaluated under the same experimental conditions. For each graph instance and each algorithm we performed 100 runs: the first run was initialized at the uniform point $(1/n, \dots, 1/n)$, and the remaining 99 were initialized at random points drawn uniformly from the probability simplex. A fixed random seed was used so that the experiments are reproducible. The maximum number of iterations was set to 1000, and the stopping tolerance for the Frank–Wolfe gap was set to 10^{-6} . At each iteration, the step size was selected using exact line search along the feasible direction generated by the corresponding method.

Since the optimization problem is solved over the continuous simplex, the final iterate is not directly a discrete clique. Therefore, a common extraction procedure was applied to the final solution of every run. First, the support of the final vector was selected using the threshold

$$\frac{1}{2n},$$

where n is the number of vertices of the graph. If the selected support was not already a clique, a greedy repair step was applied by iteratively removing vertices until a valid clique was obtained. This post-processing step was applied uniformly to all methods, so that the comparison is based on the same rule for converting continuous solutions into discrete cliques.

4.2 Performance metrics

The performance of the methods was evaluated using four criteria: the best clique size found over the 100 runs, the ratio between the obtained clique size and the best known clique size, the mean clique size across all starts, and the total computational time.

All reported clique sizes correspond to valid cliques after the common support extraction and greedy repair procedure. Therefore, the values reported below should be interpreted as the final discrete cliques obtained from the continuous Frank–Wolfe iterates, not simply as the raw numerical supports of the final vectors.

4.3 Clique quality

Table 2 reports the best clique size obtained by each method on the three benchmark instances. The best known clique size is also reported for comparison.

Table 2: Best clique sizes obtained by the three Frank–Wolfe variants.

Dataset	FW	AFW	PFW	Best known
C125-9	34	33	33	34
brock200-2	10	10	10	12
keller4	11	11	11	11

On C125-9, the classical Frank–Wolfe method recovered a clique of size 34, matching the best known value. The Away-Step and Pairwise variants both recovered cliques of size 33. On brock200-2, all three methods returned a clique of size 10, while the best known clique size is 12. This confirms that brock200-2 is the most challenging instance among the selected datasets. On keller4, all methods recovered a clique of size 11, matching the best known value.

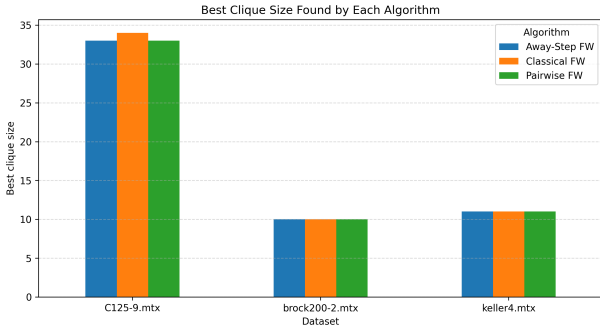


Figure 1: Best clique size found by each Frank–Wolfe variant on the three benchmark instances.

Figure 1 visually confirms that the three methods have similar performance in terms of best clique size. The main difference appears on C125-9, where FW reaches the best known clique size, while AFW and PFW remain one vertex below it.

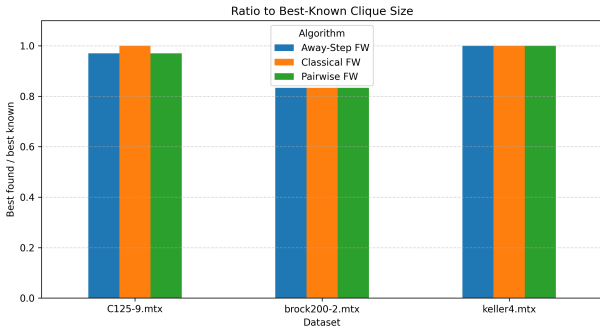


Figure 2: Ratio between the best clique size found by each method and the best known clique size.

The same information is normalized in Figure 2. A value equal to 1 means that the method recovered a clique matching the benchmark value. FW reaches this value on C125-9 and keller4, while all methods reach approximately 0.83 on brock200-2. This shows that the largest performance gap is mainly due to the difficulty of this instance.

4.4 Stability across starts

Since the problem is non-convex, the final result can depend on the initialization. For this reason, each method was run from 100 independent random starting points, and the mean clique size across all starts was recorded.

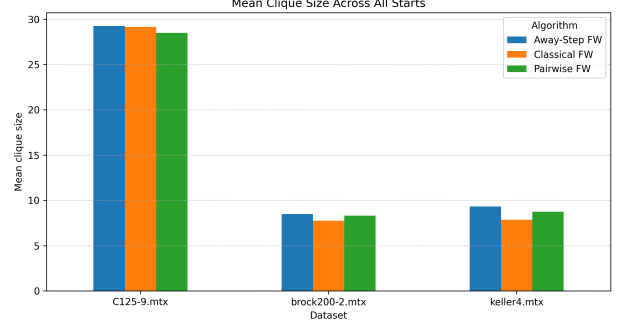


Figure 3: Mean clique size obtained across all random starts for each method and dataset.

Figure 3 shows that the average behaviour of the methods is consistent with the best-clique comparison. For C125-9, the mean clique sizes are close to the best values, indicating that the algorithms often find large cliques on this instance. For brock200-2 and keller4, the average clique sizes are smaller, showing a stronger dependence on the starting point. This confirms the usefulness of using multiple starts.

4.5 Computational time

In addition to solution quality, we compared the total computational time required by the three methods. This allows us to evaluate the trade-off between clique quality and efficiency.

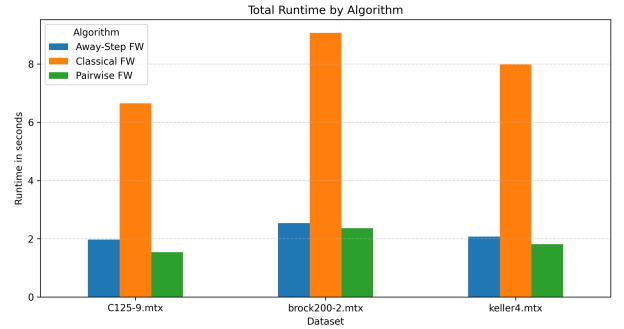


Figure 4: Total runtime required by each Frank–Wolfe variant on the benchmark instances.

Figure 4 shows that the classical Frank–Wolfe method required the largest computational time on all three datasets. The Away-Step and Pairwise variants were consistently faster, with PFW usually being the fastest or among the fastest methods. Since all methods were run with the same number of starts and stopping criteria, these results suggest that, in our implementation,

the active-set variants can provide comparable clique quality with lower computational cost.

4.6 Discussion

Overall, the three methods produced similar clique-quality results after the common extraction procedure. FW achieved the best result on **C125-9**, reaching the best known clique size, and all methods reached the best known value on **keller4**. However, none of the methods reached the best known clique size on **brock200-2**, which remained the hardest benchmark instance.

The post-processing step played an important role in the final evaluation. Since the Frank–Wolfe methods solve a continuous relaxation over the simplex, the final iterate must be converted into a discrete set of vertices. The common thresholding and greedy repair procedure ensured that all reported outputs were valid cliques.

From a computational perspective, AFW and PFW were more efficient than FW in our implementation, while maintaining competitive clique sizes. This suggests that active-set variants are attractive alternatives when computational efficiency is important.

5 Conclusions

In this project, we studied the application of Frank–Wolfe-type methods to the maximum clique problem through a continuous optimization formulation over the probability simplex. We implemented and compared three variants: the classical Frank–Wolfe method, the Away-Step Frank–Wolfe method, and the Pairwise Frank–Wolfe method. The methods were tested on three DIMACS benchmark instances: **C125-9**, **brock200-2**, and **keller4**.

The numerical results show that all three methods are able to produce valid clique solutions after applying the common support-extraction and greedy repair procedure. In terms of clique quality, the methods behaved similarly overall. The classical Frank–Wolfe method reached the best known clique size on **C125-9**, while all three methods reached the best known value on **keller4**. On the other hand, none of the methods recovered the best known clique size on **brock200-2**, confirming that this instance is the most challenging among the selected benchmarks.

The experiments also highlight the importance of using multiple random starts. Since the objective is non-convex, the final solution can depend on the initialization. Running each method from several starting points improves the probability of finding larger cliques and provides a more reliable comparison between the algorithms.

From the computational point of view, the Away-Step and Pairwise variants were faster than the classical Frank–Wolfe method in our implementation, while maintaining competitive clique sizes. This suggests that active-set variants can be effective alternatives

when computational efficiency is important.

Overall, the project shows that Frank–Wolfe methods provide a useful continuous-optimization approach to the maximum clique problem. However, the quality of the final discrete solution depends not only on the continuous optimization method, but also on the extraction procedure used to convert the final iterate into a valid clique.

References

- [1] Immanuel M. Bomze. Evolution towards the maximum clique. *Journal of Global Optimization*, 10(2):143–164, 1997.
- [2] Immanuel M. Bomze, Francesco Rinaldi, and Damiano Zeffiro. Frank–Wolfe and friends: A journey into projection-free first-order optimization methods. *4OR*, 19:313–345, 2021.
- [3] James T. Hungerford and Francesco Rinaldi. A general regularized continuous formulation for the maximum clique problem. *Mathematics of Operations Research*, 44(4):1161–1173, 2019. doi:10.1287/moor.2018.0954.
- [4] Martin Jaggi. Revisiting Frank–Wolfe: Projection-free sparse convex optimization. In *Proceedings of the 30th International Conference on Machine Learning (ICML)*, pages 427–435, 2013.
- [5] David S. Johnson and Michael A. Trick. *Cliques, Coloring, and Satisfiability: Second DIMACS Implementation Challenge*, volume 26 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*. American Mathematical Society, Providence, RI, 1996.
- [6] Simon Lacoste-Julien. Convergence rate of Frank–Wolfe for non-convex objectives, 2016. arXiv preprint arXiv:1607.00345.
- [7] Simon Lacoste-Julien and Martin Jaggi. On the global linear convergence of Frank–Wolfe optimization variants. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 496–504, 2015.
- [8] B. F. Mitchell, Vladimir F. Demyanov, and V. N. Malozemov. Finding the point of a polyhedron closest to the origin. *SIAM Journal on Control*, 12(1):19–26, 1974.
- [9] Ryan A. Rossi and Nesreen K. Ahmed. DIMACS benchmark graphs — network data repository. <https://networkrepository.com/dimacs.php>, 2015.

